

B2.2.2 Arrays and lists

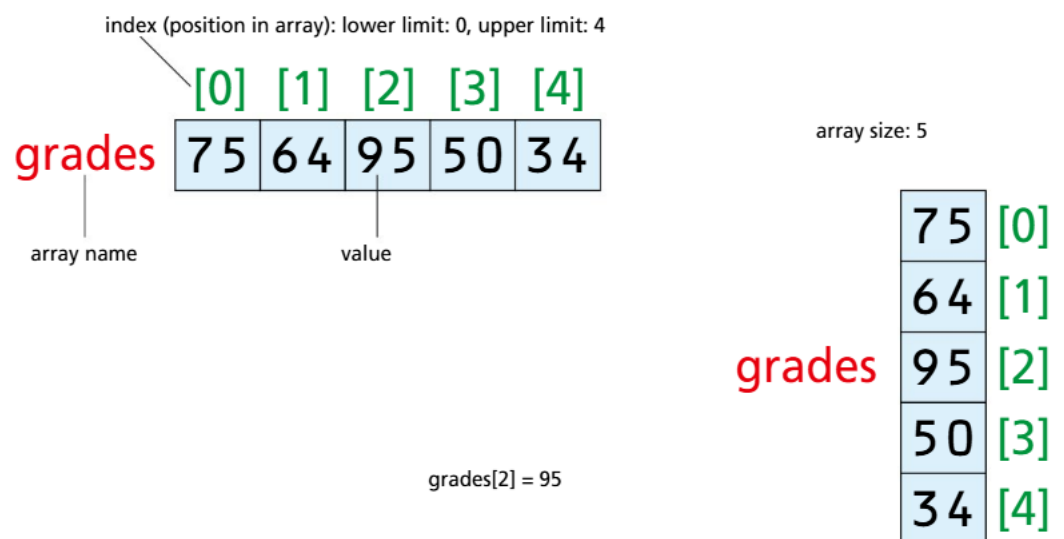
■ One-dimensional arrays (Java) and one-dimensional lists (Python)

Suppose your teacher wants to store the grades of 100 students. One way to complete this task is to use 100 variables, but this means lots of memory waste, many lines of code and a lot of effort. And what if this teacher needs to store 1000 grades?

A better solution is to store all those 100 or 1000 elements under a single name or a single identifier. The array data structure allows you to do this.

A 1D array (Java) or a 1D list (Python) is a data structure that stores elements of the same data type, under one single identifier (name). Those elements can be directly accessed by using an index (plural *indices*), where the index indicates the position in the array. Some languages indicate the first index as being the 0 element, and others allow the programmer to indicate how the items will be addressed.

For example, an array of five integers storing students' grades out of 100 is called `grades`. Graphically, arrays can be represented horizontally or vertically.



■ 1D array (list)

The array element at index 2 (the 3rd element, when counting from 0) can be directly accessed by using the name of the array followed by the index: `grades[2]`. Keep in mind that, if the first element has index 0, the last element is at index 4 (in this case, the `grades` array has only five items).

To process arrays, loops are used. Loops can be used to traverse the array; therefore, any time you perform calculations on all elements of an array, find an element, sort them, and so on, you will use loops.

One-dimensional arrays (Java)

Array declaration and initialization:

To use arrays in Java, they must be declared, and the language allows for different ways to do this. First, square brackets `[]` are used to differentiate between a variable and an array. They

can appear before or after the array name:

```
int grades []; //an array called grades of integers
int [] grades; //grades array of integers
```

After deciding where you want to place the square brackets (most often, the first example is used), you need to specify the size of the array, as it is a static data structure.

Declaration:

```
int grades [] = new int [5]; //grades array can store maximum 5
//elements
```

If the array is of integer or double or float type, and there is no further initialization of the array elements, Java automatically initializes all its elements to 0.

Initialization:

```
grades [0] = 75;
grades [1] = 64;
grades [2] = 95;
grades [3] = 50;
grades [4] = 34;
```

At this point, the elements of the array are given specific values, the value 0 being overwritten.

Another way to declare and initialize the array is like this:

```
int grades [] = new int [] { 75, 64, 95, 50, 34 };
```

But this can also be rewritten in a simpler way, like this:

```
int grades [] = { 75, 64, 95, 50, 34 };
```

Sometimes, the developer will have to allow the user to initialize the array. In such cases, you can simply declare the array and use a loop to allow the user to populate the array.

```
read = new Scanner(System.in);
int grades [] = new int [5];
for (int i = 0; i < 5; i++) {
    System.out.println("Enter a number: ");
    int value = read.nextInt();
    grades[i] = value;
}
```

The loop repeats five times, and it requires the user to enter an integer value that will be stored into the array indicated by index *i* (from 0 to 4).

If the array is of type string, it would look like this:

```
String pilots[] = {"Bob", "John", "Elvis"};
```

An array of Boolean data type can look like this:

```
Boolean passedExam[] = {true, false, true};
```

One-dimensional lists (Python)

Python replaces arrays with lists, but you can manipulate those lists to behave like arrays.

In Python, data structures don't need to be declared. Square brackets are still used to specify that the data structure used is a 1D list.

For example, the grades array will look like this:

```
grades = [75, 64, 95, 50, 34]
```

If the user wants to initialize a 1D list by using a FOR loop, then the code would look like this:

```
grades = [0]*5 #creates an array of 5 zeros
for i in range(5): #repeats 5 times
    #stores value entered at given index
    grades[i] = int(input("enter a value: "))
```

The first line in the code above specifies the initial value each element in the list will take (in this case, 0). After the *, it is the size of the list (meaning there are five 0s in this list at first).

If you were to write this line of code as `grades = [ ]`, when trying the FOR loop an error would be thrown (index out of range), as the size of the list would be none or zero, so there would not be a way to add any new elements to it. However, the following statement is completely valid:

```
grades = []
grades = [0 for i in range(5)]
```

And this is possible because the grades list is overwritten. But this could have been just like this:

```
grades = [0 for i in range(5)]
```

The purpose of the line above is to initialize the grades list to 0 for all its five elements.

The following two lists are examples of a 1D list of type string and a 1D list of type Boolean:

```
pilots = ["Bob", "John", "Elvis"]
passedExam = [True, False, True]
```

◆ **Parallel arrays:** a group of arrays of the same size, where the element at a given index in one of the arrays corresponds to another element at the same index in another array, like descriptions of a single entity.

Parallel arrays represent two or more arrays of the same size, such that the n^{th} element of the first array is related to the n^{th} element of the second array, and so on. The values stored on those arrays at a given index will be related in some form, for example defining a record data structure to store details about students. So, the first array will store the students' names, the second array their ages, the third array their addresses. When retrieving the element of position 3 from each of those arrays, you'll be able to identify the name, age and address of the third student.

PROGRAMMING EXERCISES

- 1 Construct a program that creates a 1D array or list that stores six integer values. Output true if the first element in the array is the same as the last element in the array.
- 2 Construct code that requires the user to enter a number. Use that number as the size of an array or list of integers to be inputted from the keyboard. Calculate the sum of all even numbers in the array or list; calculate the average of all its elements; and count how many of those elements are odd numbers. Display those values.
- 3 Construct a program that creates an array or list that stores five names. Copy those elements in a second array or list in reverse order. Display the second array or list.

ACTIVITY

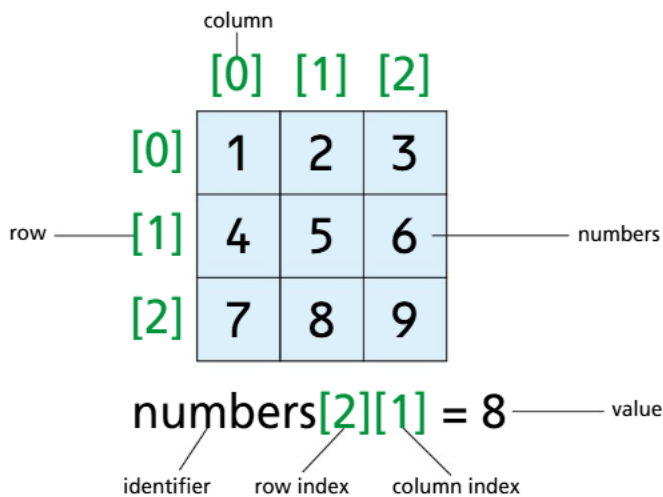
Use your answers to the programming exercises above to answer the following questions.

- 1 What data type was your output for question 1? Was it a Boolean or a string variable? Which one do you think is more appropriate?
- 2 What was the initial value of the count variable you used for question 2 to count how many array elements were odd? Why was it important to set an initial value for this variable?
- 3 Did the results for question 3 display accurately? What loops did you use to traverse the arrays? Could those loops have been replaced by other types of loops? Which are those?

Common mistake

Algorithms written to solve a problem should function correctly. When asked to construct an algorithm that finds the smallest value in a 1D array or 1D list, many candidates initialize a variable called `min` to 0 and then compare each element in the array with that one to find a smaller value. The data structure could store only positive numbers; therefore, you will not find a value smaller than 0.

Remember to initialize that variable to the first element in the array. In this case, if no other element is smaller, you have already stored the smallest one.



■ 2D array (list)

Two-dimensional arrays / lists

A two-dimensional array can be seen as a table, with rows and columns. To manipulate and traverse a 2D array, two indices are used: one for the rows and one for the columns. When writing code, rows are always first and columns second.

Look at the figure for storing numbers from 1 to 9 in a table with three rows and three columns. You will create a 2D array or a 2D list for this purpose.

Each element in a 2D array (Java) or 2D list (Python) can be directly accessed by specifying its indices (row index and column index).

Declaration

Java

```
int numbers[ ][ ] = new int [3] [3];
```

Python

```
numbers = [[0 for i in range(3)] for j in range(3)]
```

The first value of 3 indicates the number of rows, and the second one the number of columns. The value of 0 in the Python example refers to the initial value this list will be set to.

In Python, the 2D list will appear as a table with three rows and three columns, all filled with zeros, as shown in the diagram below:

	[0]	[1]	[2]
[0]	0	0	0
[1]	0	0	0
[2]	0	0	0

■ 2D array (list) filled with zeros

In Java, the 2D array will only include the structure of the table, as there are no initial values provided:

	[0]	[1]	[2]
[0]			
[1]			
[2]			

■ Empty 2D array

Manipulating 2D arrays or lists requires the use of two loops: the first one for the rows and the second one for the columns. When addressing one specific item in the array or list, this is done by using the name of the array or list followed by square brackets that include the row and column indices.

`numbers[0][2]` refers to the element in the array `numbers`, located at the intersection of the first row with the third column.

Java

```
Scanner reader = new Scanner(System.in);
int numbers[] [] = new int[3][3];
for(int i = 0; i<3;i++) {
    for(int j=0; j<3;j++) {
        System.out.println("enter a number: ");
        int value = reader.nextInt();
        numbers[i][j]=value;
    }
}
for(int i = 0; i<3;i++) {
    for(int j=0; j<3;j++) {
        System.out.print(numbers[i][j]+ " ");
    }
    System.out.print("\n");
}
```

Python

```
numbers = [[0 for i in range(3)]for j in range(3)]
for i in range (0,3):
    for j in range(3):
        value = int(input("enter a number: "))
        numbers[i][j]=value
for i in range(3):
    for j in range(3):
        print(numbers[i][j], " ", end = "")
    print("\n")
```

The first two count-controlled loops are used to traverse the array (i for the rows and j for the columns). Then the user is asked to enter a number that is stored in the variable value, which is then assigned to the array or list numbers.

The last two loops are used to traverse the array or list, so the values stored in the array are displayed on the screen.

A 2D array can also have a different number of rows and columns.

Java

```
int numbers[ ] [ ] = new int [2] [3];
```

Python

```
numbers = [[0 for i in range(2)] for j in range(3)]
```

In the example above, the array numbers will be constructed with two rows and three columns.

To fill the array with values, the following code can be used:

Java

```
int numbers[ ][ ] = new int [2] [3];
```

Python

```
numbers = [[0 for i in range(2)] for j in range(3)]
```

PROGRAMMING EXERCISES

- 1 Construct code that creates a two-dimensional array or list with three rows and three columns. Fill it with values read from the keyboard. Calculate the sum of all values and their average, and display the results.
- 2 Construct a program that creates a two-dimensional array or list with three rows and three columns. Display the sum of all elements per column. Display the average of all elements per row.
- 3 Construct code that creates a two-dimensional array or list with three rows and three columns that stores random numbers. Display the array and output the sum of all the elements on the principal diagonal. Calculate the sum of the elements on the secondary diagonal and display this value as well.
- 4 Matrix calculations are used extensively in machine learning within Computer Science. The following constraints exist to calculate a dot product: width of matrix A must match height of matrix B. Given two 2D arrays (lists) of integers that represent matrices, construct a program to calculate and solve the dot product of those two matrices.

Common mistake

Algorithms written to solve a problem should function correctly. Many students lose marks for incorrect use of indices in 2D arrays or incorrectly looping through 2D arrays, such as overlooking boundaries or not using two indices to traverse the array. Remember that you always use rows first and columns second.

"Dot product"

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix} \checkmark$$

Reflect on the most challenging parts of the programming exercises. How did you approach them? Did you seek support, did you research or did you apply any other strategies? What did you learn from this experience that you can apply to solve future problems?

■ ArrayLists in Java

An ArrayList in Java is a class that allows the use of a dynamic array to store elements, this time without a pre-defined size. ArrayLists can hold duplicate elements, they allow for random access and they maintain the insertion order.

To be able to use an ArrayList, you need to import the `java.util` package:

```
import java.util.*;
```

This will import the entire package, or you can specifically choose to import just the required class:

```
import java.util.ArrayList;
```

After this step is completed, an ArrayList can be declared, as follows:

```
ArrayList<Integer> grades = new ArrayList<Integer>();
```

Keep in mind that, when working with ArrayLists, the data type integer appears as `Integer` instead of `int`.

If the ArrayList will store text, then it would look like this:

```
ArrayList<String> pilots= new ArrayList<String>();
```

To be able to manipulate elements in an ArrayList, there are specific methods that can be used:

Method	Explanation	Code example
<code>add()</code>	Add a new element passed as an argument	<code>grades.add(75);</code>
<code>get()</code>	Access an element at a given position (specified in the brackets)	<code>grades.get(2);</code>
<code>set()</code>	Update an element at a given index (first argument is the index; the second one is the value)	<code>grades.set(2, 64);</code>
<code>remove()</code>	Delete an item at a specific index	<code>grades.remove(2);</code>
<code>size()</code>	Returns the number of elements in the ArrayList	<code>grades.size();</code>
<code>clear()</code>	Delete all elements in the ArrayList	<code>grades.clear();</code>

To traverse an ArrayList, you can use a FOR loop:

```
ArrayList<Integer> grades = new ArrayList<Integer>();
grades.add(75);
grades.add(64);
grades.add(95);
grades.add(50);
grades.add(34);
for(int i=0;i<grades.size();i++) {
    System.out.println(grades.get(i));
}
```


In the code above, the grades ArrayList of type integer is declared. Five elements are added to the ArrayList. A loop is used to traverse it (the `size()` method will return the number of items in the ArrayList to specify how many repetitions will occur), and each element is retrieved using the `get()` function and passing the corresponding index of the element that is then displayed on the screen.

Another example of traversing the ArrayList is by using a FOR-EACH loop instead:

```
ArrayList<Integer> grades = new ArrayList<Integer>();
grades.add(75);
grades.add(64);
grades.add(95);
grades.add(50);
grades.add(34);
for (int i: grades) {
    System.out.println(i);
}
```

In this example, the variable `i` is not an index, but it is an element in the grades ArrayList instead.

■ Dynamic lists in Python

A dynamic list in Python is a dynamic data structure, meaning it does not have a fixed size. Dynamic lists can store duplicate values. When creating a dynamic list, you can simply declare it as an empty list: `grades = []`

However, when wanting to insert a new element, attempting to write something like this: `grades[0] = 75` will throw an error.

To manipulate dynamic lists, specific methods can be used:

Method	Explanation	Code example
<code>append()</code>	Add a new element passed as an argument at the end of the list	<code>grades.append(75)</code>
<code>insert()</code>	Insert a new element at a given index (first argument is the index, the second is the value to be inserted)	<code>grades.insert(0,64)</code>
<code>remove()</code>	Delete the first occurrence of a given item passed as an argument	<code>grades.remove(75)</code>
<code>pop()</code>	Delete the element at the specified index	<code>grades.pop(0)</code>
<code>pop()</code>	Delete the last element if no index is given	<code>grades.pop()</code>
<code>clear()</code>	Empty the entire list	<code>grades.clear()</code>
<code>len()</code>	Return the size of the list	<code>len(grades)</code>

To loop through a dynamic list, a FOR loop can be used:

```
grades= []
grades.append(75)
grades.append(64)
grades.append(95)
grades.append(50)
grades.append(34)
for i in range(len(grades)):
    print(grades[i])
```

Another way to achieve the same is:

```
grades=[]
grades.append(75)
grades.append(64)
grades.append(95)
grades.append(50)
grades.append(34)
for i in grades:
    print(i)
```

In this case, `i` is not the index, but an element in the dynamic list.

PROGRAMMING EXERCISES

- 1 Construct programming code to create an ArrayList in Java or a dynamic list in Python that stores five colours.
 - a Construct code to insert a sixth colour after the third.
 - b Change the element at the second position to a different colour.
 - c Delete the last element.
 - d Display the new data structure.
- 2 Construct a program that creates an ArrayList in Java or a dynamic list in Python that stores three numbers.
 - a Append three more values to the end of the data structure.
 - b Display the size of the data structure.
 - c Display the first element.
 - d Store the first element into a variable.
 - e Replace all the other elements with this value.
 - f Display the new values.
- 3 Construct a program that creates an ArrayList in Java or a dynamic list in Python that stores three different numbers.
 - a Store the second value into a variable and insert it into the data structure at the end.
 - b Display the index of the first occurrence of that value.
- 4 Construct programming code that creates an ArrayList in Java or a dynamic list in Python that stores four different values.
 - a Display all the elements.
 - b Swap the values of the second and third elements.
 - c Display the new data structure.

B2.2.3 Stacks

If you were a bank cashier, you would deal with lots of coins. To easily manipulate those, the coins are categorized based on their values and stored in piles, for example a pile of 10p coins, a stack of 20p coins, another stack of 50p coins, and so on. When the cashier needs one of these coins, they would get it from the top of the pile containing the required coin value, so the stack remains intact. When the cashier needs to add a new coin to one of the piles, they would add the new coin to the pile; again, on top of it.



■ Stacks of coins

Another example would be a stack of plates: a new plate is added to the top of the stack and a plate is removed from the top of the stack, otherwise plates can break.

In Computer Science, when performing actions that work on the same principle, a specific abstract data structure called **stack** can be used. The stack works on the principle of **Last In First Out** (LIFO) or **First In Last Out** (FILO), meaning that only the top element is accessed. The operation of removing an item from the stack is called **pop**, which means taking off the top element, while the operation of adding an item to the stack is called **push**, meaning adding an item to the top of the stack.

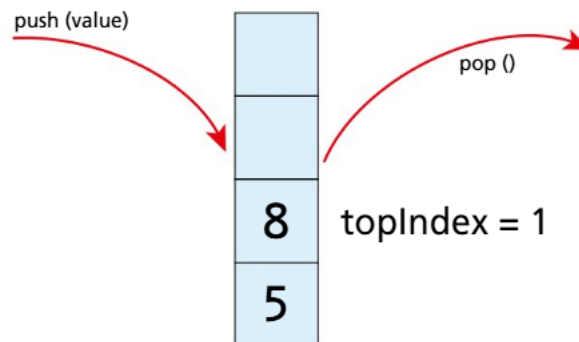
◆ **Stack**: an abstract data structure that works on the LIFO principle.

◆ **Last In First Out** or **First In Last Out principle**: the last element inserted is the first element removed.

◆ **Pop**: a method for deleting the element from the top of a stack.

◆ **Push**: a method for inserting an element at the top of a stack.

◆ **Stack pointer**: a register used to store the memory address of the last added data in a stack, or sometimes the first available address in a stack.



■ Stack

A stack could use a **stack pointer** variable to indicate the next free available slot in the stack.

A stack can be implemented as a static data structure using arrays (it would have a fixed size) or as a dynamic data structure using a linked list (which would not have a fixed size).

Stack operations

Operations performed on a stack are:

- **isEmpty**: to check whether the stack is empty; attempting to pop an item from an empty stack would throw an underflow error

```
public static boolean isEmpty() {
    if(topIndex == -1)
        return true;
    else
        return false;
}
```

topIndex = -1

def isEmpty():

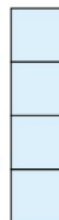
if topIndex == -1:

return True

else

return False

■ Stack operation: isEmpty



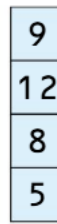
topIndex = -1

stack is empty

- `isFull`: to check whether the stack is full; attempting to push an item into a full stack would throw an overflow error

```
public static boolean isFull() {
    if(topIndex == StackSize -1)
        return true;
    else
        return false;
}
```

```
def isFull():
    if topIndex == StackSize -1:
        return True
    else:
```



`topIndex = 3`

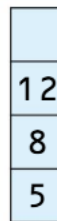
`StackSize = 4`

- Stack operation: `isFull`

- `push`: to add an item to the top of the stack

```
public static void push(int value) {
    if(isFull())
        System.out.println("Stack
overflow!");
    else {
        topIndex++;
        stack[topIndex]=value;
    }
}
```

```
def push(value):
    global topIndex
    if isFull():
        print("Stack overflow")
    else:
        topIndex += 1
        Stack[topIndex] = value
```



`topIndex = 2`

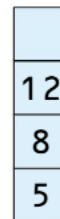
`push(9)`

- Stack operation: `push`

- `pop`: to delete an item from the top of the stack

```
public static void pop() {
    if(isEmpty())
        System.out.println("Stack underflow");
    else
        topIndex--;
}
```

```
def pop():
    global topIndex
    if isEmpty():
        print("Stack underflow")
    else:
        topIndex -= 1
```



`topIndex = 2`

`pop()`

- Stack operation: `pop`

- `peek`: to return the top element

```
public static int peek() {
    return stack[topIndex];
}
```

```
def peek():
    return Stack[topIndex]
```



`topIndex = 2`

- Stack operation: `peek`

■ Uses of stacks

Stacks are used when storing data in the order they occurred and when it might be necessary to track back to a certain point or action in the past, as they respect the LIFO principle of operation.

Stacks can be used:

- to create an **UNDO feature in games**: actions are pushed into the stack; when undoing, actions are popped
- to allow **backwards navigation on a web browser**: pressing the back button performs a pop operation; opening a new website performs a push operation
- for **interrupt handling**: when the current activity of the CPU is interrupted, the content of variables and return addresses are stored in a stack; after the **interrupt** is dealt with, those are popped from the stack to restore its activity
- to **evaluate arithmetic expressions**: when evaluating RPN (reverse polish notation) expressions, the operands are pushed on to the stack; when an operator is met, if it is a binary operator (requires two operands) two more pop operations will occur, the calculation is performed and the result is pushed back into the stack – the process will continue until the result is reached
- for **recursion**: return addresses and values of parameters are stored into a stack to help with the unwinding process.

◆ **Interrupt handling:**

handling interrupt requests.

◆ **Interrupt**: a signal sent from a device or software to request the processor's attention; the processor will stop its current activity until the interrupt has been serviced.

◆ **Recursion**: a process that uses a function or procedure that is defined in terms of itself and calls itself.

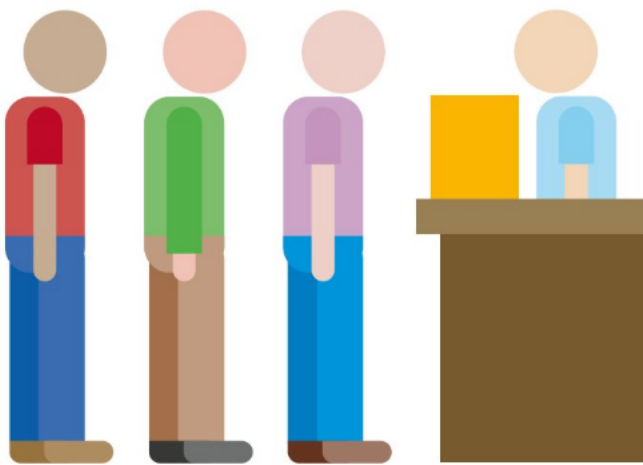
◆ **Queue**: an abstract data structure that works on the FIFO principle.

◆ **First In First Out principle**: when the first element inserted is the first element removed.

B2.2.4 Queues

A **queue** is a data structure that functions on the FIFO (**First In First Out**) principle. An example of a queue is a line of people waiting to check out at a grocery store. The first to join the queue is the first to leave. A new person who joins the line will enter at the end of the queue.

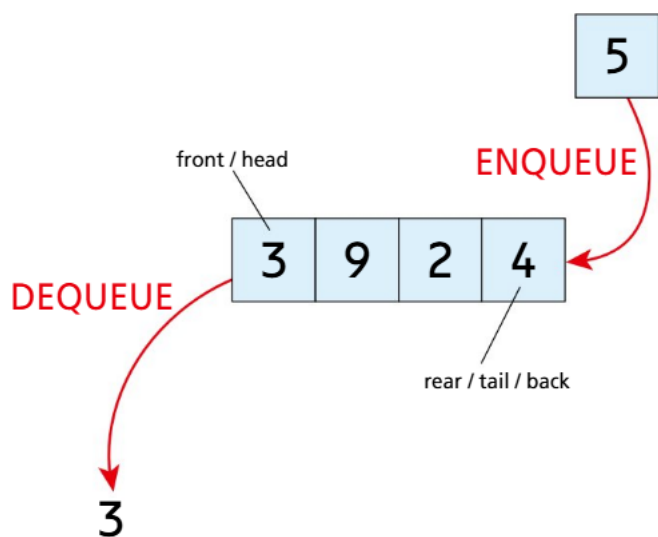
A queue uses a front pointer to show where elements will be removed from and a rear pointer to show where elements will be added to.



■ Queue

A queue can be implemented as a static data structure using arrays (it will have a fixed size, and it is possible to become full as this fixed size cannot be exceeded). Or, a queue can be implemented as a dynamic data structure, using a linked list (which would not have a fixed size).

When a queue is implemented as an array, it can often be seen and managed as a circular queue. This is in case there are elements removed from the queue, so that there is still a way to add new elements, even if the last index has been reached, as the queue will not actually be full. In a circular queue, if the rear pointer becomes equal to the front pointer after an item is added, the queue becomes full; if they become equal after an item is removed, the queue becomes empty.



■ Queue pointers

◆ **Enqueue:** a method of inserting an element at the rear of a queue.

◆ **Dequeue:** a method of deleting an element from the front of a queue.

Queue operations

Operations performed on a queue are:

- **isFull:** checking whether the queue is full; trying to **enqueue** an item into a full queue would throw an overflow error
- **isEmpty:** checking whether the queue is empty; trying to **dequeue** an item from an empty queue would throw an underflow error
- **enqueue:** adding an item to the rear of the queue
- **dequeue:** removing an item from the front of the queue
- **front:** displaying the element stored at the front of the queue.

Uses of queues

Queues are used when dealing with simulations or situations that require the first item entered to be the first item dealt with. Applications of queues include:

- **Printer queues:** Printing jobs are stored in a queue. The first printing job sent is the first to be dealt with. New printing jobs join the queue at the rear of the queue. As such, all printing jobs are dealt with on a first come first served basis.
- **Keyboard queues:** The keyboard buffer (which stores characters as they are typed) operates as a queue. As such, the first letter to be shown on the screen is the first letter typed. New characters are added to the end of the queue.
- **Simulations of real-life situations:** Checkout queues, car-washing queues, carpark-exiting queues, and so on, are all situations that require a queue. People or cars enter the queue on one side and exit it on the other side.

REVIEW QUESTIONS

- 1 Define the term "stack".
- 2 Define the term "queue".
- 3 Identify the main operations of a stack.
- 4 Identify the main operations of a queue.
- 5 Explain the FIFO and the LIFO principles and how they apply to stacks and queues.
- 6 Identify a real-life scenario where you could use a stack.
- 7 Identify a real-life scenario where you could use a queue.
- 8 Outline a situation where the use of a stack might be more efficient than the use of a queue.
- 9 Outline a situation where the use of a queue might be more efficient than the use of a stack.
- 10 Identify a scenario where you might combine a stack and a queue within the same program.
- 11 You must store a list of numbers in a particular sequence and then retrieve them in reverse order. Identify the data structure that is most suitable for this task and explain your reasoning.
- 12 Outline three uses of a stack.
- 13 Outline three uses of a queue.
- 14 Describe two characteristics of a stack.

- 15 Describe two characteristics of a queue.
- 16 Compare the use of a stack with arrays or lists.
- 17 Compare static and dynamic data structures in terms of data storage and data access.
- 18 Explain the meaning of stack overflow and stack underflow and describe when they occur.
- 19 Use a stack to evaluate the following arithmetic expression: $5 \times 2 + 3 \times 2 -$

PROGRAMMING EXERCISES

- 1 Construct a program that creates a stack that has a maximum size of seven elements. Push five words into the stack. Display the top element. Pop two words from the stack and display the top element.
- 2 Construct a program to reverse the elements from a given queue using a stack.

Common mistake

Students often lose marks by not giving responses appropriate to the keywords used in the question. For example, “compare static and dynamic data structures” is expecting you to provide both similarities and differences between the two types of data structures, rather than just differences.

ACTIVITY

Use your solutions to the programming exercises above to answer the following questions.

- 1 Why is the maximum size of the stack important? What happens if this maximum size is exceeded?
- 2 How would you change the solution to Exercise 2 if you were asked to reverse the elements of a stack using a queue structure instead?

Common mistake

When required to state applications of a data structure, such as a queue or stack, students often lose marks for not being clear enough to gain the marks. For example, saying that stacks are used to undo would not be enough to gain any marks.

Be specific and explain a clear situation where an undo feature is needed, such as implementing an undo feature in games: actions are pushed on to the stack and, when the previous action is needed, the undo feature is used to pop the top action from the stack.

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ B2.4.1 Describe the efficiency of specific algorithms by calculating their Big O notation to analyse their scalability
- ▶ B2.4.2 Construct and trace algorithms to implement a linear search and a binary search for data retrieval
- ▶ B2.4.3 Construct and trace algorithms to implement bubble sort and selection sort, evaluating their time and space complexities
- ▶ B2.4.4 Explain the fundamental concept of recursion and its applications in programming (HL)
- ▶ B2.4.5 Construct and trace recursive algorithms in a programming language (HL)

B2.4.1 Big O notation

Complexity analysis of algorithms includes time and space analysis. Time complexity analysis refers to how long an algorithm will take to run or how many steps an algorithm will take to run, while space analysis refers to how much memory space it takes to run the algorithm.

■ Time complexity

Consider the following algorithm:

Java

```
for(int i = 0; i<5;i++) {  
    System.out.println("hello");  
}
```

Python

```
for i in range (0,5):  
    print("hello")
```

How many times will this loop repeat? The variable *i* will take values from 0 to 4 included; therefore, it is simple to say that the algorithm will repeat five times. If we are to replace the 5 with *n*, the algorithm will repeat *n* times and the time complexity will be of the order of *n*. This is written as $O(n)$, which is known as **Big O notation**. The time it takes to run the algorithm is approximately proportional to *n*. The larger the *n*, the more accurate the approximation; therefore, when calculating the Big O notation, the worst-case analysis is considered (the larger the *n*, the better).

◆ **Big O notation:** used to find the upper bound (worst-case scenario or the highest possible amount) of the growth of a function; the longest time or space required to turn the input into output.

Java

```
for(int i = 0; i<n; i++) {  
    System.out.println("hello");  
}
```

Python

```
for i in range (0,n):  
    print("hello")
```